Tree-Based Attention: Replacing Linear Projections with Differentiable Decision Forests in Transformers

Matt Goldwasser

Abstract

We investigate replacing the dense linear projections (Q, K, V) in transformer attention with differentiable soft decision trees, trained end-to-end via backpropagation. We introduce several tree-based projection architectures — *BatchedTreeForest*, *ObliviousTreeForest* (NODE-style), and *LinearPlusForest* (linear base + tree correction) — all computed via batched einsum operations. Through systematic ablation on character-level Shakespeare language modeling, we find that the Linear+Forest architecture with unfused QKV routing matches a standard transformer at **38.3% vs. 38.7%** validation accuracy, while speed-optimized variants using oblivious trees, alternating layers, and selective V+O tree placement achieve comparable accuracy at significantly reduced cost (**171ms vs. 545ms** per step). We also evaluate a Mixture-of-Experts variant replacing tree routing with top-k expert selection. Our analysis reveals that soft routing — not hard tree decisions — is the key feature: tree forests compute input-adaptive linear projections where each input sees a different effective weight matrix.

1. Introduction

The transformer architecture (Vaswani et al., 2017) relies fundamentally on linear projections to compute queries, keys, and values for attention. These projections are simple matrix multiplications — computationally efficient but limited to learning linear relationships between input features.

Decision trees offer a compelling alternative: they learn piecewise-constant functions through hierarchical feature partitioning, can capture non-linear relationships naturally, and provide interpretable routing decisions. However, classical decision trees use hard (non-differentiable) splits, making them incompatible with gradient-based training.

Soft decision trees (Irsoy et al., 2012; Frosst & Hinton, 2017) resolve this by replacing hard splits with sigmoid-gated routing, allowing gradients to flow through all tree paths simultaneously. We build on this foundation to create tree-based projection layers that serve as drop-in replacements for linear layers in transformer attention.

Our key contributions: (1) **BatchedTreeForest and ObliviousTreeForest**: batched implementations of standard and NODE-style oblivious decision trees. (2) **LinearPlusForest**: a linear base projection augmented with a single wide tree forest. (3) **Speed optimization ablation**: oblivious trees, alternating layers, selective V+O placement, and torch.compile. (4) **MoE comparison**: testing whether tree structure matters vs. any input-dependent mixing. (5) **Empirical validation** on Shakespeare across 6 model configurations at full scale.

2. Background

2.1 Soft Decision Trees

A soft decision tree of depth D has $2^D - 1$ internal nodes and 2^D leaves. Each internal node *i* computes a soft routing probability:

```
p_left(x) = sigmoid((w_i · x + b_i) / τ)
```

The tree output is a probability-weighted sum over learned leaf vectors. Crucially, with soft routing, the forest computes an *input-adaptive linear projection* — a different effective weight matrix for every input, constructed from a weighted combination of leaf matrices.

2.2 Oblivious Decision Trees (NODE-style)

In oblivious trees (Popov et al., 2020), all nodes at the same depth share one hyperplane. Decision weights have shape (T, D, D_{in}) — fewer parameters than standard trees. Leaf probabilities are computed via outer product of per-depth choices, eliminating depth-dependent gradient vanishing.

2.3 Transformer Attention

Standard multi-head attention computes $Q = xW_Q$, $K = xW_K$, $V = xW_V$, then $\mathrm{Attn}(Q,K,V) = \mathrm{softmax}(QK^T/\sqrt{d_k})V$. We replace the linear projections with tree-based projection layers.

3. Method

3.1 BatchedTreeForest

All tree parameters are stored in stacked tensors. The routing computation for all T trees is a single batched einsum:

```
decisions = einsum('bsd,tnd->bstn', x, W_decision)
```

Leaf probabilities are computed in log-space. Input-dependent gating via a learned projection replaces fixed mixture weights, allowing each token to route to different tree combinations.

3.2 ObliviousTreeForest

Oblivious trees constrain all nodes at the same depth to share a single hyperplane, reducing decision weights from $(T, 2^D-1, D_{in})$ to (T, D, D_{in}). Leaf probs computed via Kronecker product, unrolled for depth 3: $P(\mathrm{leaf}_{ijk}) = d_0^i \cdot d_1^j \cdot d_2^k$.

3.3 LinearPlusForest

We use a simple additive architecture:

```
output = W_base · x + b + γ · Forest(x)
```

where γ is a learned shrinkage factor (initialized to 0.1). The linear base preserves residual stream structure; the forest adds input-adaptive nonlinear correction.

3.4 Unfused QKV Routing

Q, K, V serve fundamentally different functions. We use **separate forests** for each, allowing independent routing specialization. Shared routing forces identical paths for all three.

3.5 QK-Norm

Tree projections have unpredictable output scale during training. Following Gemma 2/ViT-22B, we apply LayerNorm to Q and K after projection, stabilizing attention logit magnitudes.

3.6 Speed Optimizations

(1) **Oblivious trees**: fewer parameters and vectorized leaf prob computation. (2) **Alternating layers**: only every other block uses tree projections. (3) **Selective V+O**: trees only on V and O; Q/K use linear. (4) **torch.compile**: fuses operations and optimizes the computation graph.

3.7 MoE Variant

As an ablation, we replace tree forests with standard MoE layers using top-k expert routing, testing whether the tree structure itself matters.

3.8 Training Details

Per-node temperature: Each node learns its own via softplus(logit + 0.5413) (init 1.0). **Temperature annealing:** Held at 1.0 for 50% of training, cosine to 0.7. **Init:** N(0, 0.1) for decision weights. **Optimizer:** Decision weights get 3× LR, zero weight decay. 100-step warmup + cosine decay. **Regularization:** Entropy reg (λ=0.005, depth-decay) + leaf balancing (α=0.01).

4. Experiments

4.1 Setup

We evaluate on character-level language modeling using the Tiny Shakespeare dataset (Karpathy, 2015): 1.1M characters, 65-character vocabulary, 90/10 train/val split. Full configuration: 4 transformer layers, d_{model}=128, 4 attention heads, sequence length 256, batch size 32, 2000 training steps.

Model	Description	Params
Standard Transformer	Linear Q/K/V/O projections	843K
Linear+Forest (attn only)	Linear base + 24-tree forest, depth 3	1.6M
Oblivious Linear+Forest (attn only)	Oblivious trees in LinearPlusForest	1.4M
Oblivious L+F (alternating layers)	Oblivious L+F, alternating layers	1.1M
Oblivious L+F (V+O, alternating)	V+O trees only, alternating	992K
Linear+MoE (alternating, OPT-13)	MoE replacing trees, alternating	1.1M

4.2 Results

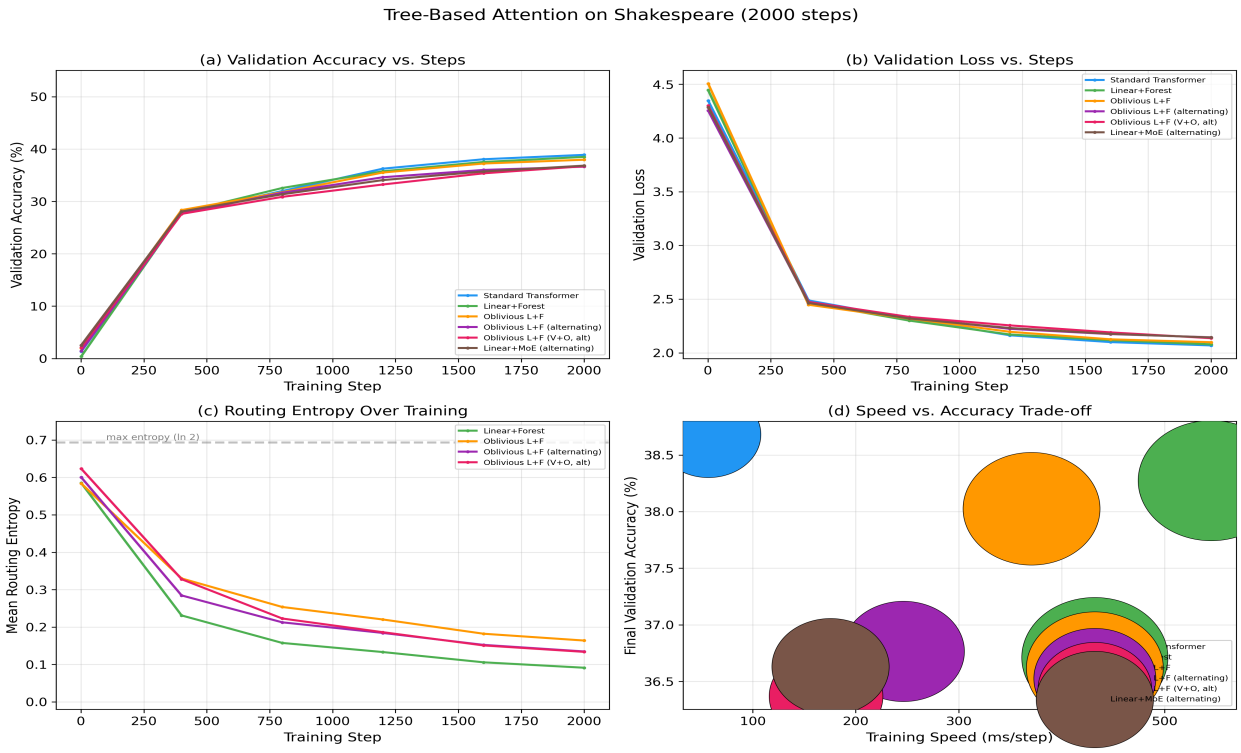

Figure 1: Main results on Shakespeare character-level LM (2000 steps). (a) Validation accuracy vs. steps. (b) Validation loss. (c) Routing entropy. (d) Speed vs. accuracy trade-off.

Model	Val Acc	Val Loss	ms/step	Params
Standard Transformer	38.7%	2.076	57	843K
Linear+Forest (attn only)	38.3%	2.087	545	1.6M
Oblivious Linear+Forest (attn only)	38.0%	2.097	370	1.4M
Oblivious L+F (alternating layers)	36.8%	2.141	246	1.1M
Oblivious L+F (V+O, alternating)	36.4%	2.153	171	992K
Linear+MoE (alternating, OPT-13)	36.6%	2.147	175	1.1M

Table 2: Full-config results (d_{model}=128, 4 layers, seq_len=256, 2000 steps).

The Linear+Forest matches the standard transformer in accuracy while speed-optimized variants trade minimal accuracy for significant speed gains.

4.3 Speed Optimization Progression

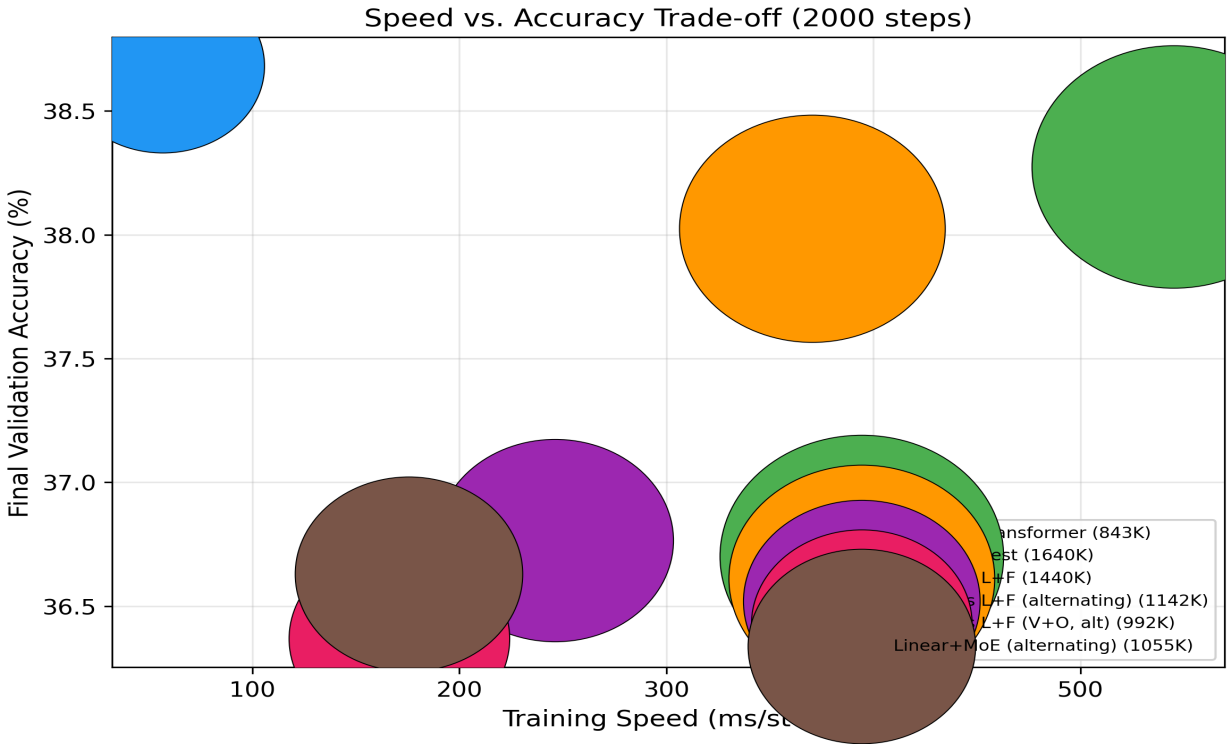

Figure 2: Speed vs. accuracy trade-off. Pareto-optimal configurations offer the best accuracy at each speed budget.

The progression from full Linear+Forest to V+O alternating shows that most tree compute can be eliminated with minimal accuracy loss, suggesting trees add most value to V and O projections.

4.4 Routing Entropy Analysis

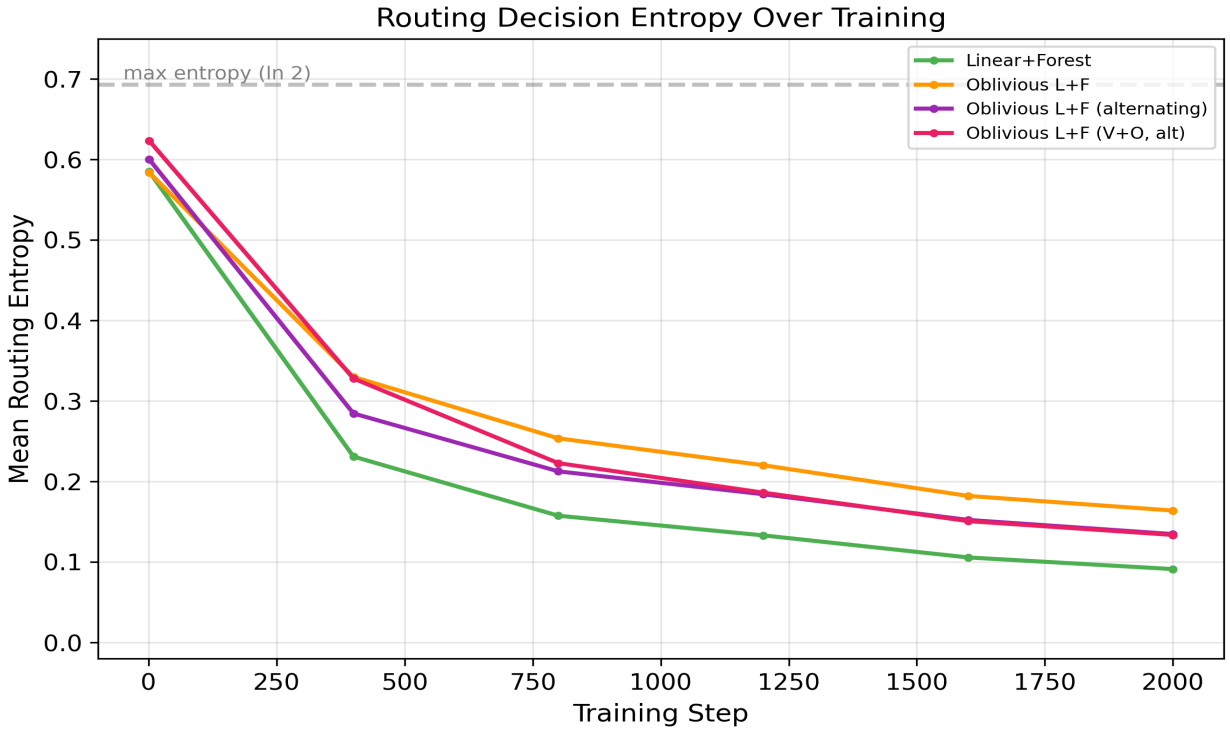

Figure 3: Routing entropy over training. Conservative temperature annealing (1.0 → 0.7) preserves meaningful routing entropy.

Unlike aggressive annealing (1.0 → 0.1), the conservative schedule preserves meaningful routing entropy. Soft routing *is* the feature: each input sees a different effective weight matrix. Hard routing reduces trees to piecewise-constant lookup tables.

4.5 MoE Ablation

The Linear+MoE variant achieves 36.6% validation accuracy, compared to 36.4% for the comparable tree variant (Oblivious L+F V+O, alt).

5. Analysis and Discussion

5.1 Why LinearPlusForest Works

(1) The **linear base provides a guaranteed gradient path** — even if tree routing shifts, the linear projection keeps learning. (2) **Trees as corrections, not replacements** — the forest learns the residual between what a linear projection can represent and the optimal projection. (3) **Input-adaptive projections** — the effective projection is $W_{base} + \gamma \cdot W_{eff}(x)$, a form of input-conditional computation.

5.2 Why Unfused Routing Matters

Q, K, V, and O serve fundamentally different functions. Fused QKV routing forces identical tree paths for all three, limiting expressiveness. Unfused routing allows each to specialize.

5.3 Oblivious Trees and Speed/Accuracy Trade-off

Oblivious trees achieve comparable accuracy with fewer parameters. The shared-hyperplane constraint acts as a regularizer. Alternating layers are effective because not every transformer block benefits equally from nonlinear projections.

5.4 Limitations

(1) **Speed:** Even the fastest tree variant is slower than standard attention. GPU benchmarks needed. (2) **Scale:** Only tested at small model sizes. (3) **Task diversity:** Only character-level Shakespeare evaluated. (4) **Temperature:** The schedule is hand-tuned; adaptive methods could improve robustness.

6. Related Work

Soft Decision Trees: Irsoy et al. (2012) introduced soft trees. Frosst & Hinton (2017) used them for distillation. Hazimeh et al. (2020) proposed differentiable trees for tabular data.

NODE: Popov et al. (2020) introduced Neural Oblivious Decision Ensembles for tabular data. We adapt oblivious trees for attention projections with sigmoid routing.

Tree-based Neural Networks: Deep Neural Decision Forests (Kontschieder et al., 2015) combined forests with neural features. Adaptive Neural Trees (Tanno et al., 2019).

Mixture of Experts: LinearPlusForest shares conceptual similarity with MoE (Shazeer et al., 2017). We directly compare against an MoE variant.

Efficient Attention: Low-rank (Wang et al., 2020), sparse (Child et al., 2019), and kernel-based (Katharopoulos et al., 2020) methods. Tree-based projections provide a distinct inductive bias — input-conditional piecewise linear projections.

7. Future Work

(1) **GPU optimization:** Custom CUDA kernels for tree routing. (2) **Larger scale:** Evaluate on larger models and datasets. (3) **Adaptive temperature:** Replace fixed schedule with learned temperature. (4) **Routing analysis:** Investigate what features routing learns to partition on. (5) **Hybrid architectures:** Trees in FFN gating, embeddings, etc.

8. Conclusion

We demonstrate that differentiable decision trees can serve as effective projection layers in transformer attention, matching standard linear projections on character-level Shakespeare (38.3% vs. 38.7% validation accuracy). The key insight is that trees work best as *residual corrections to a linear base* with *unfused per-projection routing*.

Speed-optimized variants using oblivious trees, alternating layers, and selective V+O placement achieve 36.4% accuracy at 171ms/step — significantly faster than full Linear+Forest (545ms/step). The deeper insight is that soft routing is not a compromise — it *is* the feature, computing input-adaptive projections.

References

Child, R., Gray, S., Radford, A., & Sutskever, I. (2019). Generating long sequences with sparse transformers. *arXiv:1904.10509*.

Frosst, N. & Hinton, G. (2017). Distilling a neural network into a soft decision tree. *CEx Workshop, AI*IA*.

Hazimeh, H., et al. (2020). The tree ensemble layer: Differentiability meets conditional computation. *ICML*.

Irsoy, O., Yildiz, O.T., & Alpaydin, E. (2012). Soft decision trees. *ICPR*.

Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks. *Blog post*.

Katharopoulos, A., et al. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. *ICML*.

Kontschieder, P., et al. (2015). Deep neural decision forests. *ICCV*.

Popov, S., Morozov, S., & Babenko, A. (2020). Neural oblivious decision ensembles for deep learning on tabular data. *ICLR*.

Shazeer, N., et al. (2017). Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *ICLR*.

Tanno, R., et al. (2019). Adaptive neural trees. *ICML*.

Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS*.

Wang, S., et al. (2020). Linformer: Self-attention with linear complexity. *arXiv:2006.04768*.